Teleprompter Script

AI-Driven Malware Detection via Windows Event Log Engineering
in Smart Grid Control Systems

Contents

Slide 1 — Title

Good morning and welcome. Today I will present our framework for artificial intelligence-driven malware detection using Windows Event Log engineering in smart grid control systems. This work addresses a critical gap at the intersection of industrial control system security and machine learning, specifically targeting the under-studied layer of host-level Windows audit logs within operational technology environments. *[pause]* We will walk through the problem motivation, our threat model, the synthetic data generation approach, feature engineering pipeline, model design, and experimental results, closing with practical deployment considerations. *[click to next slide]*

Slide 2 — Agenda

The agenda has six major sections. We begin with the motivation—understanding why smart grid control systems are attractive targets and why Windows Event Logs are both a regulatory requirement and an untapped detection surface. We then review the related literature on log anomaly detection and ICS intrusion detection systems. After that, I will present our formal threat model, followed by the data generation and feature engineering methodology. We will look at experimental results including the ablation study, and conclude with a discussion of real-world deployment constraints. *[click to next slide]*

Slide 3 — Smart Grid Cyber Threat Landscape

The electric grid is no longer air-gapped. The integration of IP connectivity, programmable logic controllers, and Windows-based SCADA systems has created a convergence zone where IT and operational technology interact daily. This expanded attack surface has been exploited in high-profile incidents—Stuxnet demonstrated in 2010 that Windows hosts embedded in nuclear centrifuge control loops were viable entry points, and the 2015 Ukrainian grid attacks showed that adversaries could traverse the IT/OT boundary to cause physical power outages. *[click]* Regulatory frameworks have responded: NERC Critical Infrastructure Protection standard CIP-007 mandates event logging on all applicable cyber assets, and IEC 62351 specifies audit trail requirements for power system communications. *[click]* What these mandates produce is terabytes of Windows Event Log data on SCADA hosts—but this data is almost never used for automated intrusion detection. *[click]* That gap is the central motivation for this work. The figure on the right shows the architecture we target.

Slide 4 — Related Work: Log Anomaly Detection

The system log anomaly detection literature has produced impressive results over the last decade. Drain, published at ICWS 2017, introduced an online log-parsing algorithm using a fixed-depth tree structure that extracts stable log templates from high-velocity streams in logarithmic time. DeepLog, from CCS 2017, was perhaps the most influential contribution: it modelled sequences of log event keys using LSTM networks, treating each next-key prediction as a classification problem and flagging deviations as anomalies. LogAnomaly extended this by incorporating semantic word vectors derived from log templates, capturing both sequential and quantitative anomalies. *[pause]* The common limitation of all these works is their focus on distributed computing infrastructure—HDFS, Linux syslog, OpenStack—rather than Windows systems in industrial control environments. None of them exploit the semantic structure of Windows Security Event IDs, which encode precise information about authentication, process creation, and object access. Our approach fills this gap.

Slide 5 — Related Work: ICS Intrusion Detection

In the ICS domain, Carcano and colleagues proposed a multidimensional critical-state analysis that captures safe operating envelopes as polytopes in the joint measurement-command space. Goldenberg and Wool built deterministic finite automata from Modbus/TCP protocol specifications, achieving zero false positives on a power utility testbed. The SWaT dataset from Singapore University of Technology and Design provided the

first publicly available ICS security benchmark, though it captures physical process measurements and network packets rather than Windows host audit logs. *[click]* A 2014 ACM Computing Surveys paper by Mitchell and Chen confirmed that host-level intrusion detection is rare in the cyber-physical systems literature—the research community has concentrated on the network and process layers. *[click]* Our contribution is therefore novel: we are the first to target Windows Event Logs at Purdue model Levels one through three within a smart grid context.

Slide 6 — Threat Model

Our adversary has three goals: gain persistent access to SCADA hosts, escalate privileges to the system or domain-administrator level, and ultimately execute a disruptive or destructive payload—this could be rogue command injection into a PLC or simply service disruption to deny operator visibility. *[click]* In terms of capabilities, we assume the adversary has already compromised at least one Level 3 IT host, for instance through spear phishing or a supply chain attack. They possess valid or cracked domain credentials and have network connectivity across the IT/OT boundary through a misconfigured DMZ or a legitimate remote access tunnel. *[click]* From this starting point, we model four specific attack categories grounded in the MITRE ATT&CK for ICS framework: lateral movement through credential brute forcing, persistence through malicious service installation, privilege escalation through offensive tool execution, and reconnaissance through rapid object access bursts. Each of these produces a distinguishable pattern in the Windows Event Log that our feature engineering pipeline is designed to capture.

Slide 7 — Synthetic Event Log Generator

Because labelled Windows Event Log data from real ICS environments is not publicly available—due to privacy, liability, and classified-information constraints—we built a synthetic generator. The generator produces structured CSV records with eight fields: timestamp, event ID, source, user, hostname, description, label, and session ID. Benign sessions simulate realistic operator workflows with a logon event, variable object-access and process-creation activity, and a terminal logoff, with inter-event gaps drawn from a uniform distribution representing normal operator think-time. *[pause]* Our dataset contains six thousand labelled sessions—five thousand benign and one thousand malicious—comprising just over fifty-four thousand individual events. The five-to-one class ratio reflects a realistic operational environment where malicious sessions are rare. Smart grid hostnames follow the naming conventions you would find in a real utility: SCADA-HMI-01, RTU-CTRL-03, eng_station, and historian. The figure on the right shows the full data pipeline from generation through model evaluation.

Slide 8 — Session-Level Feature Engineering

The core of our approach is the session-level feature engineering pipeline. Rather than processing individual events, we group events by session and extract a compact twenty-four-dimensional feature vector. The first group captures event frequency counts for each of thirteen Windows Event IDs plus a total count—this distinguishes attack patterns that rely on specific event types. The second group—and the most discriminative as we will see—captures time-delta statistics: the mean, standard deviation, minimum, and maximum of inter-event gaps in seconds. Reconnaissance attacks, which burst dozens of object-access events within thirty seconds, produce dramatically different time-delta profiles than normal operator sessions. *[pause]* The third group captures cardinality—unique sources, users, and hostnames—as a lateral-spread indicator. Group four uses rare-event flags for event IDs appearing in fewer than five percent of benign training sessions. And group five computes Shannon entropy over the event-ID distribution within each session, capturing the randomness or monotony of event sequences. Together these five groups produce a compact, interpretable vector with no feature scaling required.

Slide 9 — Model: Random Forest

We selected Random Forest as the primary classifier for four reasons. *[click]* First, it handles the heterogeneous mixture of integer counts, continuous statistics, and binary flags in our feature vector without requiring normalisation—a practical advantage for deployment pipelines. *[click]* Second, the `class_weight=balanced` setting compensates for the five-to-one class imbalance by assigning per-sample weights inversely proportional to class frequencies, without requiring synthetic oversampling. *[click]* Third, feature importance scores derived from mean decrease in Gini impurity provide an inherently interpretable ranking, directly supporting the ablation analysis. *[click]* Fourth, and critically for operational deployment, inference requires only a fixed number of tree traversals per session—our measurements show sub-millisecond latency per session on commodity hardware, well within the ten-millisecond operational requirement. The training protocol uses Randomized Search CV over twenty hyperparameter configurations with five-fold stratified cross-validation, maximising macro F1.

Slide 10 — Classification Results

The results table compares our Random Forest against three baselines: Logistic Regression, Decision Tree, and RBF Support Vector Machine. All baselines use the same twenty-four-dimensional feature set and the same class-imbalance compensation. Logistic Regression achieves a macro F1 of zero-point-nine-four-three, Decision Tree reaches zero-point-nine-seven-two, and SVM reaches zero-point-nine-six-one. The Random Forest achieves a perfect macro F1 of one-point-zero and a ROC-AUC of one-point-zero on the held-out test partition. The false positive rate of zero means no benign sessions were misclassified, which is critical for operational acceptance—false alarms in an energy control room impose real cognitive and operational costs. This result confirms that the synthetic data are fully discriminable by the engineered feature set under the controlled generative model. Performance on real-world operational logs with label noise and evasion behaviours is expected to be lower, and that evaluation constitutes future work.

Slide 11 — ROC Curve and Feature Importances

[show figure] The left panel shows the ROC curve for the Random Forest on the test set—a perfect curve hugging the top-left corner, confirming the AUC of one. The right panel shows the top-ten feature importances. The most important feature is the logoff count—event ID four-six-three-four—because benign sessions reliably end with a logoff while attack sessions often do not. The next four most important features are all temporal: mean inter-event gap, maximum gap, gap standard deviation, and sequence entropy. This pattern is consistent with the burst-timing signatures of reconnaissance and lateral movement attacks. Event frequency counts and cardinality measures appear lower in the ranking, contributing incrementally but not decisively. This ranking directly informs the ablation study on the next slide.

Slide 12 — Ablation Study

The ablation study systematically removes one feature group at a time and re-evaluates macro F1. The results confirm what the importance ranking suggested. Removing Group 2, the time-delta statistics, causes the largest drop: macro F1 falls from one-point-zero to zero-point-eight-three-one—a drop of zero-point-one-six-nine. This is a dramatic degradation and makes intuitive sense: burst timing is the primary signature of reconnaissance attacks. *[click]* Removing Group 5, sequence entropy, causes the second-largest drop to zero-point-nine-four-seven. Event counts and cardinality contribute modestly, and rare-event flags have the smallest impact on synthetic data—though we expect their value to increase on real operational logs where attack events may be rarer and noisier. The key practical takeaway is that if a deployment scenario cannot collect all five feature groups—for instance, due to network latency constraints on timestamp resolution—time-delta statistics should be prioritised above all else.

Slide 13 — Deployment Considerations

Real-world deployment raises four significant challenges. *[click]* First, class imbalance: the five-to-one ratio in our dataset is already challenging, but operational environments may see five-hundred-to-one or higher ratios where malicious sessions are extremely rare events. Techniques such as SMOTE oversampling, cost-sensitive learning, or reformulating detection as anomaly scoring against a benign-only model should be evaluated as the class ratio approaches operational reality. *[click]* Second, concept drift: Windows Event Log distributions change over time as software is updated, organisations restructure, and operational patterns shift. Periodic retraining on sliding windows of recent data and integration with online drift detectors such as ADWIN are recommended. *[click]* Third, GDPR compliance: Windows Event Logs may contain personally identifiable information—usernames, hostnames, IP addresses. Our session-level feature aggregation is inherently privacy-preserving since it produces only statistical summaries, but the raw log collection must be governed by pseudonymisation and retention policies aligned with both NERC CIP-007 and GDPR. *[click]* Fourth, latency is not a concern: we measured zero-point-three-four milliseconds median per session, making the framework compatible with standard SIEM integration patterns.

Slide 14 — Conclusion

To summarise: we have presented the first open-source framework specifically targeting Windows Event Log-based malware detection in smart grid industrial control systems. The framework consists of a synthetic event log generator covering four MITRE ATT&CK ICS attack categories, a twenty-four-dimensional session-level feature engineering pipeline, and a class-imbalance-robust Random Forest classifier. On the synthetic benchmark, the framework achieves perfect discrimination—macro F1 of one-point-zero and ROC-AUC of one-point-zero. The ablation study establishes that temporal features are the most discriminative, consistent with the burst-timing signatures of the attack patterns modelled. Inference latency is sub-millisecond, confirming operational feasibility. *[pause]* The open-source nature of the framework directly addresses the data availability barrier that has historically limited ICS security research. Future directions include evaluation on real ICS event logs from controlled red-team exercises, extension to multi-class MITRE ATT&CK technique attribution, online learning for concept drift adaptation, and adversarial robustness evaluation.

Slide 15 — References

[This slide displays automatically. No spoken content required. Remain silent or offer to discuss any reference in detail during Q&A.]

Slide 16 — Questions

Thank you for your attention. I am happy to take questions on any aspect of the work—the threat model, the feature engineering choices, the evaluation methodology, or the deployment considerations. *[pause and make eye contact with the audience] [if no immediate questions: "Perhaps I can start with the question I am most often asked, which is whether the perfect accuracy on synthetic data translates to real-world performance—the honest answer is that we do not know yet, and that evaluation is our highest-priority future work."]*